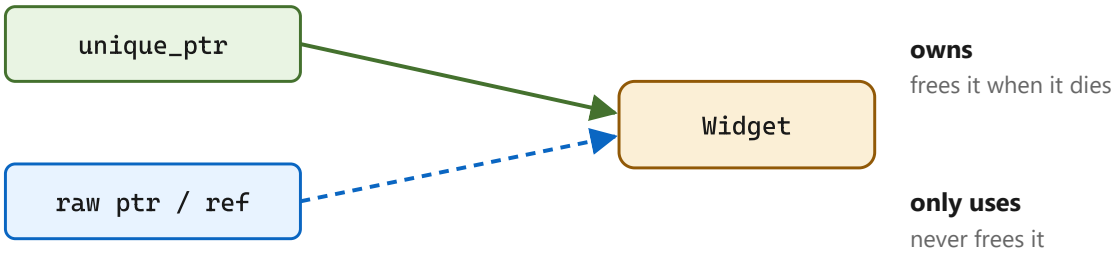
unique_ptr: Exclusive Ownership

How `unique_ptr` makes ownership explicit: creating one, why it can be moved but not copied, how to pass and return it, deleters, arrays and polymorphism, and the usual mistakes. Builds on RAII and Move Semantics.

Ownership: who frees it?

Every object created with `new` must be freed exactly once. The **owner** is the code responsible for freeing it. With raw pointers the owner is a convention in someone's head; smart pointers write it into the **type**, so the compiler and the reader both know.



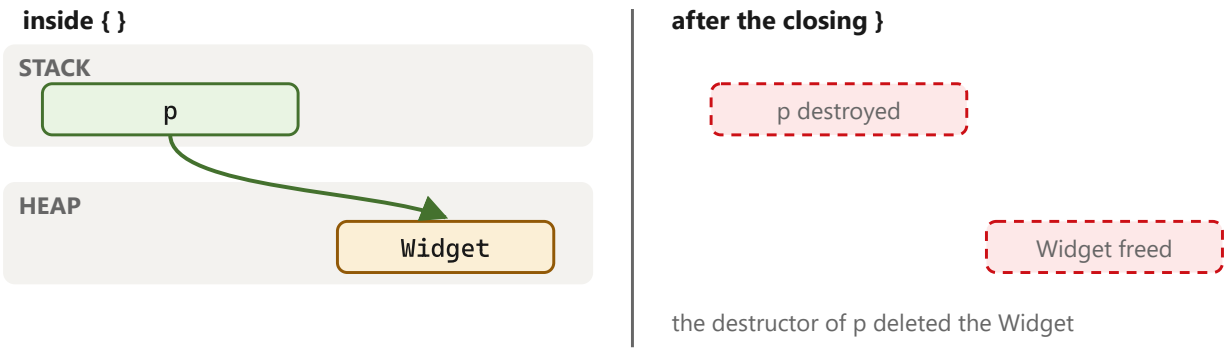
Green: the owner. Dashed blue: a user that must not outlive the owner.

TYPE	OWNERSHIP	COPYABLE?	COST
`std::unique_ptr<T>`	exclusive: one owner	**no**, only movable	same as a raw pointer
`std::shared_ptr<T>`	shared: reference counted	yes (bumps the count)	control block, atomic count
`std::weak_ptr<T>`	none: observes a `shared_ptr`	yes	small
raw pointer / reference	none: "I use it, I do not own it"	yes	none

DEFAULT A plain value member or local first. Then `unique_ptr` when the object must live on the heap (polymorphism, large, optional). `shared_ptr` only for truly shared ownership (next lesson).

Creating a `unique_ptr`

A `unique_ptr` is a small object on the stack that holds the address of a heap object. When the `unique_ptr` is destroyed, its destructor deletes the heap object. That is RAII: you cannot forget to free it, even if the function returns early or throws.



The heap object lives exactly as long as its owner.

```
{
    // preferred over `new Widget( ... )`
    auto p = std::make_unique<Widget>(1, "a");
    // use it like a pointer: -> and *
    p->id;
// Widget destroyed here, automatically
}
```

Compiled with a `Widget` that prints from its constructor and destructor: it printed "built", then "in scope", then "destroyed" at the closing brace.

KEY POINT Why `make_unique`? No naked `new`, the type is written once, and before C++17 `f(unique_ptr<A>(new A), g())` could leak when `g()` threw between the two steps.

Move-only: handing over ownership

Two owners of one object would free it twice, so a `unique_ptr` **cannot be copied**. It can be **moved**: the object stays where it is and ownership passes to the new `unique_ptr`, which leaves the old one empty (`nullptr`).



A move copies one pointer and nulls the source.

```
auto a = std::make_unique<Widget>(2, "b");
// auto c = a;                  // ERROR: unique_ptr cannot be copied
auto b = std::move(a);          // OK: b owns the Widget, a is null
```

OPERATION	ALLOWED?
copy (`auto c = a;`)	no: compile error
move (`auto b = std::move(a);`)	yes: `a` becomes null
return from a function	yes: moved out automatically
store in a `std::vector`	yes, with `push_back(std::move(p))` or `push_back(make_unique<T>( ... ))`

Passing and returning

The parameter type tells the reader what a function does with the object.

A FUNCTION THAT…	TAKES
only uses the object	`const Widget&` (or `Widget*` if it may be null): never the smart pointer
takes over ownership	`std::unique_ptr<Widget>` by value: the caller must `std::move`
gives ownership to the caller	returns `std::unique_ptr<Widget>` (a factory)

```
// factory: sole ownership to the caller
std::unique_ptr<Widget> makeWidget(int id);
// just uses: pass the object, not the owner
void use(const Widget& w);
// takes ownership
void take(std::unique_ptr<Widget> w);

auto w = makeWidget(3);
// w still owns it
use(*w);
// Widget destroyed when take() ends; w is null
take(std::move(w));
```

Compiled: `use` printed "using 3" and left `w` set; after `take(std::move(w))` the Widget was destroyed inside the call and `w` was null.

Deleters, arrays, polymorphism and the member functions

- **Zero overhead.** With the default deleter, `sizeof(unique_ptr<T>)` equals `sizeof(T*)`. Compiled: 8 and 8; with a stateless lambda deleter still 8, with a function-pointer deleter 16.
- **Custom deleter** for anything that is not `delete`, such as a file handle:

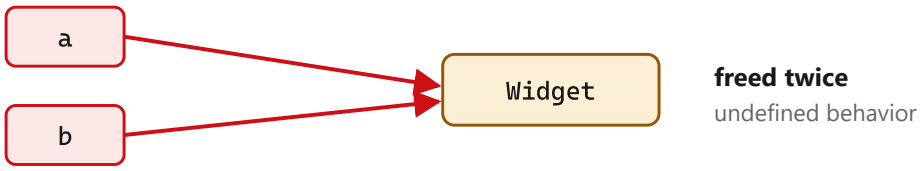
```cpp
auto closer = [](std::FILE* f) { if (f) std::fclose(f); };
std::unique_ptr<std::FILE, decltype(closer)> file(std::fopen("data.txt",
"r"), closer);
// the file is closed when `file` goes out of scope
```

- **Arrays.** `std::make_unique<int[]>(3)` uses `delete[]` and zero-initializes. Prefer `std::vector` unless you need a fixed block.
- **Polymorphism.** A `unique_ptr<Base>` holding a `Derived` needs a **virtual destructor** on `Base`, as with any `delete` through a base pointer. Compiled: `base→who()` printed "Derived".

CALL	WHAT IT DOES
`p.get()`	the raw pointer; ownership stays with `p`
`p.release()`	gives up ownership and returns the raw pointer: **you** must delete it
`p.reset()`	destroys the object now (or `reset(q)` takes over another)
`if (p)`	true when it owns something

Common mistakes

```
Widget* raw = new Widget( … ); unique_ptr<Widget> a(raw), b(raw);
```



Never give the same raw pointer to two owners.

MISTAKE	WHAT GOES WRONG	DO INSTEAD
two `unique_ptr`s from one raw pointer	double delete	one owner; move it
using `p` after `std::move(p)`	null dereference	treat a moved-from pointer as empty
`delete p.get();`	the `unique_ptr` deletes it again	never delete what you got with `get()`
a `unique_ptr` to a stack object	it deletes memory it never allocated	only `new` or `make_unique` objects
copying one into a function parameter	compile error	`std::move`, or pass `const T&`

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Creating a unique_ptr	Effective Modern C++	Item 18, Use std::unique_ptr for exclusive-ownership resource management	136 to 143
	Effective Modern C++	Item 21, Prefer make_unique and make_shared to new	157 to 165
Move-only: handing over ownership	C++ Move Semantics	Chapter 13 Move-only types	213 to 218